

Better Loops 与智能体（agent）编排：从二十个终端到更好的循环

Peter Steinberger (AI Engineer World's Fair 2026)

2026 年 7 月 8 日



视频作者/频道：AI Engineer / Peter Steinberger

发布日期：2026 年 7 月 1 日

视频时长：原视频 08:36:51；本主题 00:48:09–00:54:35

主题类型：keynote

主题编号：04

视频链接：<https://www.youtube.com/watch?v=htM02KMNZnk>

原视频入口时间：t=2889s

目录

导读：本讲的中心问题	2
1 从二十个终端到管理器 (manager)：旧工作流的问题	2
1.1 旧工作流把人变成了人肉操作系统	2
1.2 从配对到管理：角色边界发生了变化	3
1.3 为什么这一步是更好的循环的入口	4
1.4 本章小结	4
2 形成闭环的三项基础设施变化	4
2.1 第一项变化：服务端压缩让长期任务可靠运行	4
2.2 第二项变化：协调让一个线程 (thread) 能创建并引导项目	4
2.3 第三项变化：自动化让同一个管理器被事件唤醒	5
2.4 三项变化如何合成闭环	5
2.5 本章小结	6
3 瓶颈 (bottleneck) 迁移：从 token 和算力到注意力 (attention)	6
3.1 瓶颈为什么会移动	6
3.2 tokens、compute、attention 的对照	7
3.3 注意力成为瓶颈后的工作方式变化	8
3.4 本章小结	8
4 内层执行循环 (inner execution loop) 与外层决策循环 (outer decision loop)	8
4.1 从盯着代码流动到审阅意图实现	9
4.2 issue 到 PR 的内层执行流程	9
4.3 双循环边界如何保护注意力	10
4.4 本章小结	11
5 长期运行的管理器：跨线程、宿主机/机器 (host/machine) 与工作入口	11
5.1 真实例证：Paul Solt 的定期唤醒工作流	11
5.2 线程与宿主机/机器解耦	12
5.3 跨入口交互：从应用会话到随处可 steer 的智能体	13
5.4 本章小结	13
6 总结与延伸	14
6.1 演讲的最终命题：设计承载系统	14
6.2 PDF 的压缩结论：三项实践启发	14
6.3 延伸：从工具清单转向工作流架构	15

导读：本讲的中心问题

本 PDF 的结论是：Peter Steinberger 所说的更好的循环（better loops）要求先重设责任边界和循环质量，再决定终端、工具与智能体并行度。旧 workflow 让人充当调度器、路由器和临时记忆；新的 workflow 让长期运行的统筹层维持上下文、分配执行任务、响应外部事件，并在需要判断时把 PR、差异、构建或其他证据带回给人。这样，人的认知资源被保留给方向设定、风险审阅和高层决策。

证据边界

正文以英文原始字幕、主题切片画面和已检查的截图为主证据。中文自动字幕存在编码损坏风险，仅作为素材存在记录。用户给定时间边界为 00:48:09–00:54:35，内容校准以 Peter Steinberger 的发言语义为准。

1 从二十个终端到管理器（manager）：旧 workflow 的问题

Peter Steinberger 的开场先展示一张多屏工作站照片：桌面上同时打开多个终端、编辑器和监控窗口，人坐在中间等待每个任务完成，再继续给下一个智能体派活。这个画面给出本节的核心结论：旧 workflow 把人推到调度、路由和记忆三种系统职责的中心，智能体数量越多，人的认知负担越重。

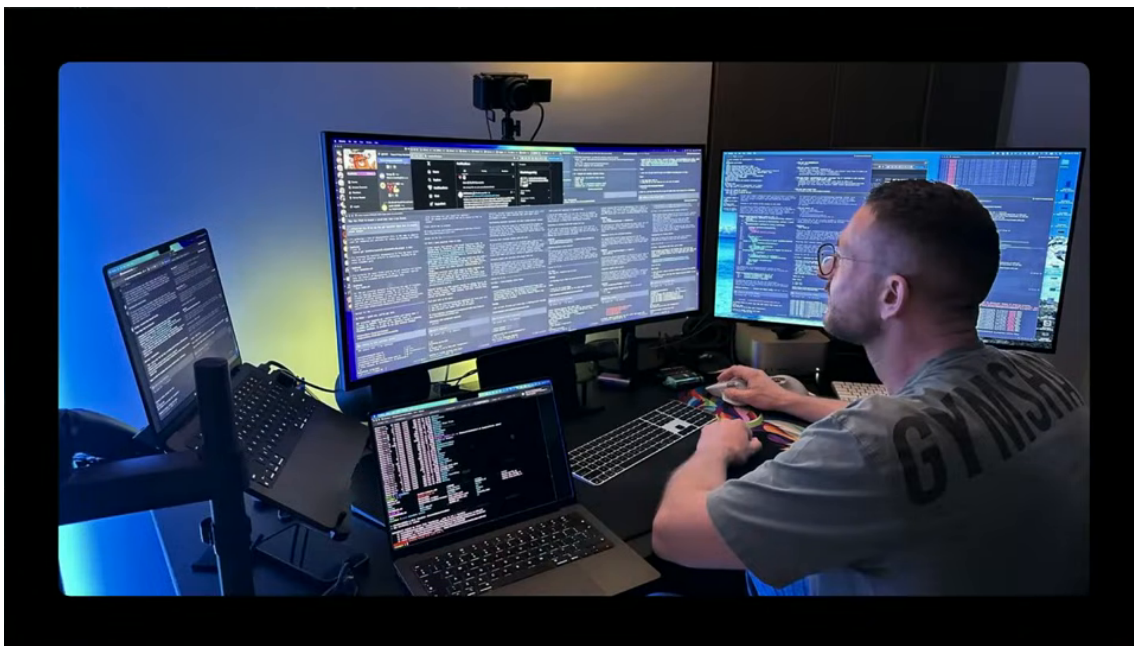


图 1：画面中可见多块屏幕、多个终端窗口和代码面板，直观呈现 speaker 所说的“同时管理十多个 terminal windows”。它支撑的机制结论是：当人需要等待、检查、切换并继续引导每个智能体时，人本身就成了工作流的调度层。¹

1.1 旧 workflow 把人变成了人肉操作系统

在 00:48:35--00:48:47，speaker 说自己曾经同时处理十个以上终端窗口，等待某一个结束后再去 steer the agent，并继续 cue new work。这里的关键观察是“等待”：多个智能体确实可以同时

¹ 视频画面时间区间：00:48:25–00:48:59。

跑，但每个智能体的完成、阻塞、需要新指令、需要人工判断，都要回到同一个人的处理队列里。这一点可以拆成三项隐藏职责：

- **调度**：人决定哪个终端先看、哪个任务先补指令、哪个结果先收敛。
- **路由**：人把一个窗口里的发现转移到另一个窗口，或者把某个结果继续分配给下一个任务。
- **记忆**：人记住每个终端的目标、当前状态、上下文约束和下一步意图。

speaker 对这段经历的回顾很直接：当时以为自己在 orchestration，实际是在 polling（轮询）。也就是说，人并没有拥有一个稳定的编排层，只是在反复查看每个窗口有没有变化。视觉上像高效率，机制上是把状态管理压到人的短期记忆里。

旧工作流的真实成本

多终端并行只扩大了执行面，没有自动生成统筹层。只要完成状态、后续派活和跨任务记忆仍由人维护，人的认知资源就会成为所有智能体共同争抢的共享资源。

1.2 从配对到管理：角色边界发生了变化

speaker 接着给出角色变化：一开始是和一个智能体配对；终端变成十个后，就像在管理十个 direct reports；现在默认方式变成和一个长期运行的管理器对话，由管理器把工作分给团队。这里的管理器指长期运行、持有上下文、分派任务并向人类返回决策材料的统筹智能体。它是 workflow 中的统筹层，避免被理解为组织职级。

在这个新结构里，工作智能体（worker）承担具体执行步骤，例如调查、实现、测试和整理结果。普通智能体仍可以执行任务、参与审查或响应指令；工作智能体强调其从属于管理器，并面向某个被分派出来的具体任务。人仍能在困难任务上直接下沉到某个工作智能体中配对，但默认路径已经变成“人管理管理器，管理器管理多个智能体”。

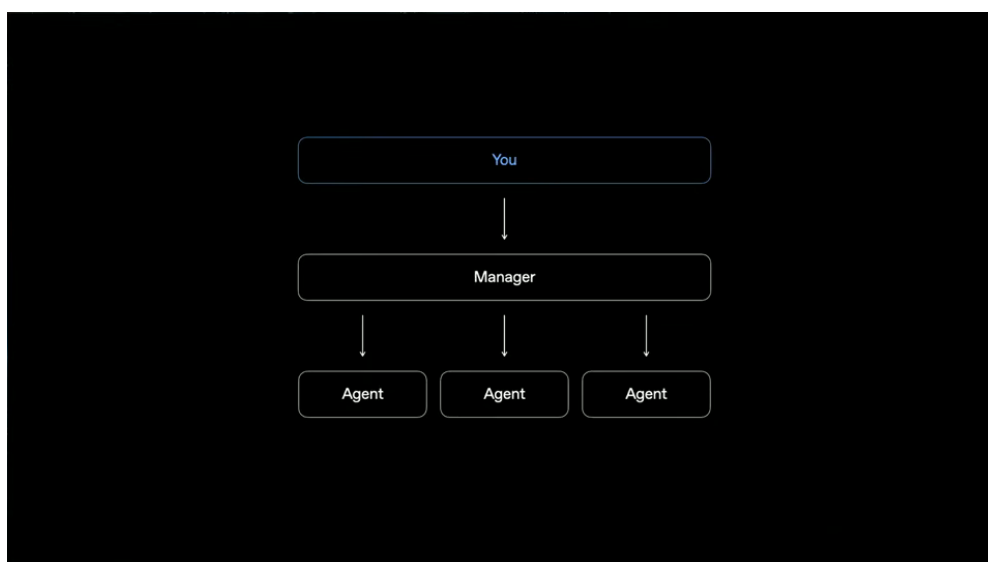


图 2: 图中结构从 You 指向 Manager，再由 Manager 分派到多个 Agent，清楚展示“人 -> 管理器 -> 智能体团队”的分派树。它支撑的机制结论是：人的直接控制面缩小，管理器承担跨任务统筹与分派。²

²视频画面时间区间：00:49:11–00:49:34。

1.3 为什么这一步是更好的循环的入口

更好的循环要求工作流拥有一个能维持目标、状态和下一步动作的中间层。旧模式下，人要记住“哪个终端做到哪里了”；新模式下，管理器要知道“哪些工作正在推进、哪些结果需要汇总、什么时候该向人返回决策材料”。

这个变化可以用一个操作系统类比理解：旧模式里，人同时扮演任务调度器、事件循环和内存；新模式里，管理器承担持续统筹，人把精力留给方向、例外和最终判断。这样并没有消除人的责任，只是把人的责任从逐条轮询移到更高层的决策位。

常见误读：把管理器当作普通岗位

本主题中的管理器是长期运行的统筹智能体。它的职责是持有上下文、分派工作、回收结果并在需要时请求人类决策。若把它理解成普通组织头衔，就会漏掉它作为软件工作流基础设施的含义。

1.4 本章小结

本节说明了旧工作流的根本问题：智能体可以变多，人的认知资源和短期记忆没有随之扩展。多终端模式把人放在调度、路由和记忆的中心，因此看起来像 orchestration，实际常常是 manual polling。转向管理器后，工作流开始拥有一个长期统筹层；人可以继续设定方向和处理难题，日常分派和状态维护则交给管理器与工作智能体承担。更好的循环从这里开始：先减少人对每个终端的逐条干预，再把人放回真正需要判断的位置。

2 形成闭环的三项基础设施变化

上一节说明了为什么旧工作流会把人压成调度层。本节回答下一个问题：为什么现在可以转向管理器结构。speaker 给出的答案很短，只有三项变化：服务端压缩（server-side compaction）、协调（coordination）和自动化（automation）。三者合在一起，让长期任务能保留状态、分派工作，并在事件发生时继续推进。

2.1 第一项变化：服务端压缩让长期任务可靠运行

服务端压缩指由服务端维护和压缩长任务上下文，使长期任务可靠运行。它解决的是“任务不能长跑”的问题。旧模式高度依赖 fresh sessions，因为每个会话像一个临时容器：上下文容易耗尽，历史容易丢失，重开会话又需要人重新交代背景。服务端压缩把长期任务的目标、约束、进度和重要记忆保留下来，让管理器有机会持续工作。

这一点和普通摘要不同。普通摘要常常只是把一段聊天压短；服务端压缩在这里服务于长期任务可靠性。它的工程意义是：人不用围绕 fresh sessions 设计工作习惯，管理器也不用每次从零开始恢复现场。

2.2 第二项变化：协调让一个线程（thread）能创建并引导项目

协调指一个线程（thread）创建、组织并引导合适项目或子任务的能力。这里的线程是工作承载单元，承载上下文、任务进展和切入点。它的作用在于让管理器把一个目标拆成可执行项目，并把不同子任务推进到合适位置。

如果服务端压缩回答“上下文如何留下来”，协调回答“留下来的上下文如何被用来组织工作”。没有协调，长期上下文只是一个很长的记事本；有了协调，管理器才能把目标拆成项目，把项目交给工作智能体，把执行结果重新聚合回来。

2.3 第三项变化：自动化让同一个管理器被事件唤醒

自动化指外部事件发生后能够唤醒同一个管理器继续工作的机制。它解决的是“谁来重新启动循环”的问题。旧模式依赖人反复轮询终端；新模式下，触发器（triggers）可以在事件发生时唤醒同一个管理器。例如任务完成、外部系统有新状态、某个检查需要处理时，管理器可以恢复上下文并继续推进。

这也是触发器与简单提醒之间的边界。提醒只是告诉人“该看了”；触发器面向管理器，让系统本身继续进入下一步。人仍然保留最终判断权，但日常唤醒动作不再全部压到人的手动检查上。



图 3: 推文写着“you shouldn’t be prompting coding agents anymore”以及“designing loops that prompt your agents”。它支撑本节的机制结论：工作重点从反复提示单个智能体，转向设计能够持续提示、分派和唤醒智能体的循环。³

2.4 三项变化如何合成闭环

speaker 在 00:49:54--00:50:03 把结果压缩成三个词：持久上下文（persistent context）、委派（delegation）和触发器。对应关系很清楚：

- 服务端压缩带来持久上下文，让长期运行的管理器能保留目标、约束、进度和记忆。
- 协调支撑委派，让管理器把具体工作交给工作智能体或其他智能体执行。
- 自动化提供触发器，让外部事件发生时同一个管理器继续推进。

这三项构成一条因果链：上下文留下来，工作才能分出去；工作分出去，结果才需要回收；结果或外部事件出现，系统才需要被触发继续运行。到这里，loop 才成立。它表现为一个可恢复、可分派、可触发的任务系统。

³视频画面时间区间：00:49:31–00:50:03。

闭环的最低条件

一个可用的管理器循环至少需要三件事：持久上下文保证任务不会丢失，委派保证执行可以分散到多个智能体，触发器保证事件发生后系统能继续推进。缺少任一项，循环都会退回人工轮询。

2.5 本章小结

本节把 speaker 的三项基础设施变化拆成一条 workflow 链条：服务端压缩让长期任务可靠，协调让线程能够组织项目和子任务，自动化让同一个管理器在事件发生时被唤醒。三者共同生成持久上下文、委派和触发器，也就是更好的循环的基本形态。读者需要保留的判断是：更好的循环应被理解为围绕长期上下文、任务分派和事件触发重新设计的软件工程 workflow。

3 瓶颈 (bottleneck) 迁移：从 token 和算力到注意力 (attention)

这一节的结论很直接：一旦更好的循环开始运转，系统最稀缺的资源会继续迁移。Peter 用一句英文压缩了这层机制：The bottleneck keeps moving. 这里的瓶颈 (bottleneck) 指当前最限制 workflow 规模和效率的资源或环节；它曾经表现为 token 预算，也曾经表现为机器算力，到了多智能体并行工作的阶段，主要表现为人类注意力 (attention)。

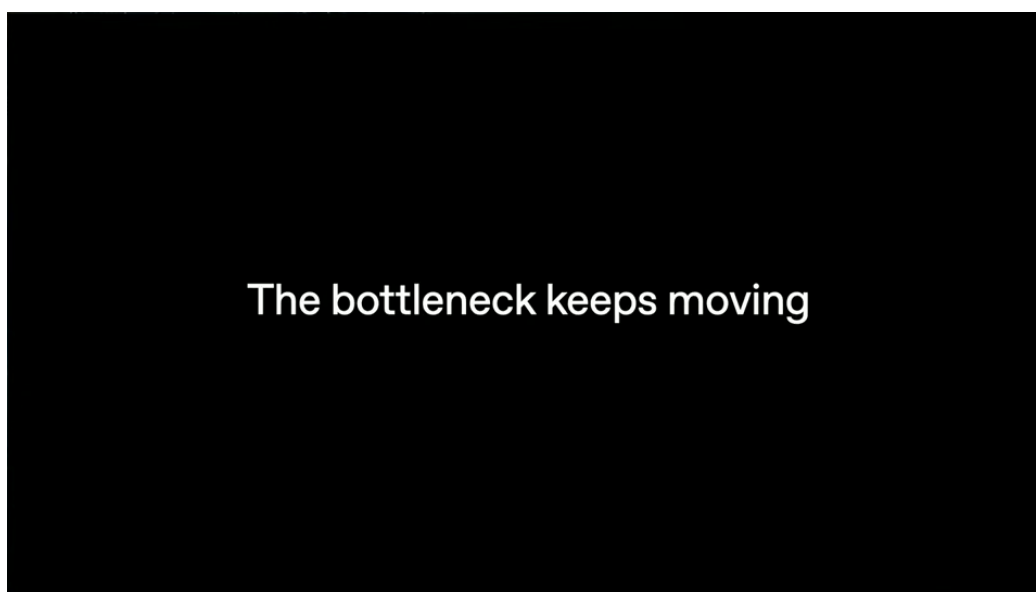


图 4: 画面只保留一句判断：瓶颈会继续迁移。它把本节从“更多资源”转向“资源类型变化”的问题。⁴

3.1 瓶颈为什么会移动

在旧的单一智能体体验里，开发者最容易感到受限的是 token。上下文窗口、可输入材料、可保留历史和可生成内容都占用 token；当 token 紧张时，任务很容易被切成 fresh sessions，管理器能保留的背景变少，持久上下文也会变薄。Peter 说自己通过加入 OpenAI 缓解了这个问题，同时立

⁴视频画面时间区间：00:50:05–00:50:17

刻承认这种策略不具备普遍扩展性。这句话的教学价值在于提醒读者：某个个体获得更多 token 配额，并不等于整个工作流架构解决了资源约束。

token 压力缓解后，约束转向算力（compute）。多个线程同时运行，多个智能体同时执行代码、跑测试、做审查，本地 MacBook 会成为热源和噪声源。Peter 提到的缓解方式是使用测试机器（test boxes），让智能体在单独机器上运行测试。这一步把执行负载从个人电脑搬到更合适的宿主环境，说明算力压力可以通过机器数量、机器类型和任务放置策略来缓解。

真正困难的迁移发生在第三层：注意力。注意力是人类用于判断、审阅、设定方向和处理异常的认知资源。它不像 token 可以买更多配额，也不像算力可以增加测试机器。它更接近会议桌上的座位：座位数量有限，放进低价值议题，就会挤掉真正需要判断的议题。

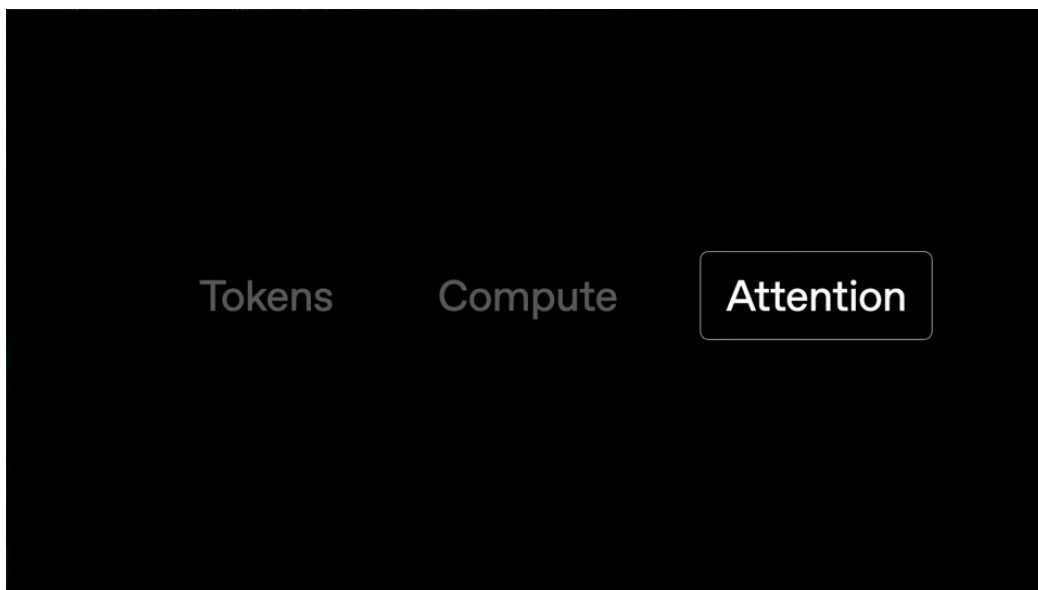


图 5: 画面把 Tokens、Compute、Attention 放在同一条约束链上，并突出 Attention，说明当前更稀缺的是人类判断资源。⁵

3.2 tokens、compute、attention 的对照

三者的差异集中在扩展方式和剩余风险，数量计算在原演讲中没有出现。下表用工程工作流的语言比较三类约束。

⁵ 视频画面时间区间：00:50:13–00:51:07

约束层	限制的对象	常见缓解方式	剩余风险
Tokens	能装进上下文的材料、历史和指令	更长上下文、服务端压缩、减少 fresh sessions	上下文变长后，错误材料和含混目标也会被长期保留
Compute	并行执行、测试、构建和审查的吞吐量	使用测试机器、分离本地与远端执行、把任务放到合适宿主环境	并发越高，失败状态、日志和结果越需要协调
Attention	人类判断、审阅、方向设定和异常处理	让管理器汇总 PR、issue、diff、运行结果和审查材料	注意力无法简单增加，低价值监控会挤占高价值判断

表 1: tokens、compute、attention 三类约束的工程含义对照。

核心判断

更好的循环的价值不只是让智能体多跑几步。更关键的价值是把人的注意力保留给少数判断点：是否值得做、是否符合目标、是否可以批准、是否需要调整方向。

3.3 注意力成为瓶颈后的工作方式变化

当注意力成为瓶颈时，开发者的主要技能随之变化。过去的核心动作是及时打断智能体、纠正方向、补充上下文；现在更重要的动作是判断注意力应该花在哪里。Peter 在这一段给出的答案是：停止继续增加终端数量，把人从持续盯守中移出来，让管理器把中间过程压缩成可审阅的决策材料。

这里有一个隐藏依赖：管理器必须已经具备足够稳定的持久上下文、委派能力和触发器。缺少这些基础条件，注意力转移会变成失控放任；具备这些条件后，人可以把审阅节奏从“看每一步”调整成“看关键交付物”。这也是本节和下一节的连接点：下一节将把这种转移落实为双循环边界。

3.4 本章小结

本节解释了资源瓶颈的三次迁移：token 限制上下文，算力限制并发执行，注意力限制人类判断。token 和算力都有相对直接的扩容路径；注意力的扩容空间很小，必须通过 workflow 设计来保护。更好的循环把智能体并行执行带来的中间噪声交给管理器处理，让人类只在少数高价值位置投入注意力。

4 内层执行循环（inner execution loop）与外层决策循环（outer decision loop）

这一节把上一节的注意力问题落到可执行边界：智能体负责内层执行循环（inner execution loop），人类负责外层决策循环（outer decision loop）。这两个循环的差别决定了开发者每天应该看什么、批准什么、留下什么备注，以及把哪些中间细节交给管理器和工作智能体处理。

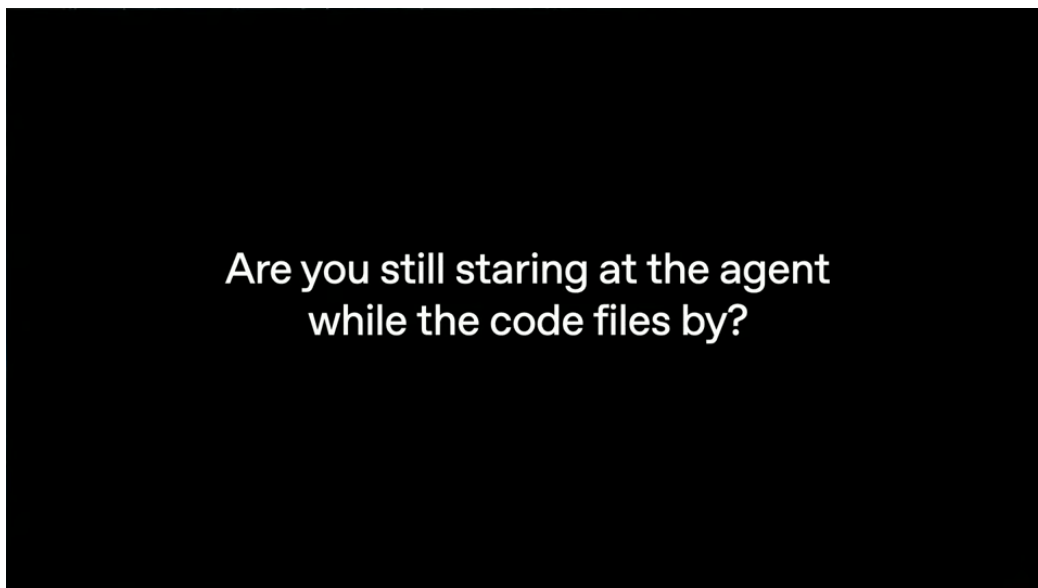


图 6: 画面提出一个操作层问题: 开发者是否还在盯着智能体生成代码。它直接承接上一节的注意力瓶颈。⁶

4.1 从盯着代码流动到审阅意图实现

Peter 先承认“看着代码飞过去”很酷, 也指出早期模型阶段这种盯守具有必要性。早期智能体容易朝错误方向展开, 开发者需要看到偏航、按下 `escape`、重新 `steer`。模型能力提升后, 最新一代模型更擅长理解意图 (`intent`), 持续观看每条中间输出的收益下降。

这句话背后的工程假设很重要: 人类仍然负责判断意图和结果是否匹配, 智能体负责把意图展开成实现步骤。人类持续消费每条中间消息, 会把注意力花在低层执行轨迹上; 人类审阅被管理器整理后的 PR、原始 `issue`、`proposed diff`、运行结果和测试状态, 注意力就集中到决策材料上。

常见误解

减少盯守仍然保留责任。人的责任从逐行监控转为定义目标、检查结果和批准变更。工作流质量的核心指标, 是管理器能否返回足够完整的证据包, 让人用审阅结论驱动下一轮决策。

4.2 `issue` 到 PR 的内层执行流程

Peter 的例子从一个开源项目 `issue` 开始。这个例子有第四步, 刚好对应图中的四个节点:

1. **Issue filed**: 有人提交 `issue`, 外部事件成为触发点。
2. **Worker created**: 管理器醒来, 把 `issue` 对照项目 `goals`、`notes` 和 `vision`, 判断是否值得处理; 如果匹配, 就创建工作智能体。
3. **Agents work**: 工作智能体调查问题、实现变更、运行测试, 另一个智能体可以审查结果。
4. **Review and approve**: 管理器在需要人类介入时返回 PR、原始 `issue`、`proposed diff`, 必要时还包括视频或可通过 VNC 进入的运行中构建; 人类审阅一次、留下备注、批准或要求修改, 检查通过后循环继续并落地。

⁶视频画面时间区间: 00:51:07–00:51:39

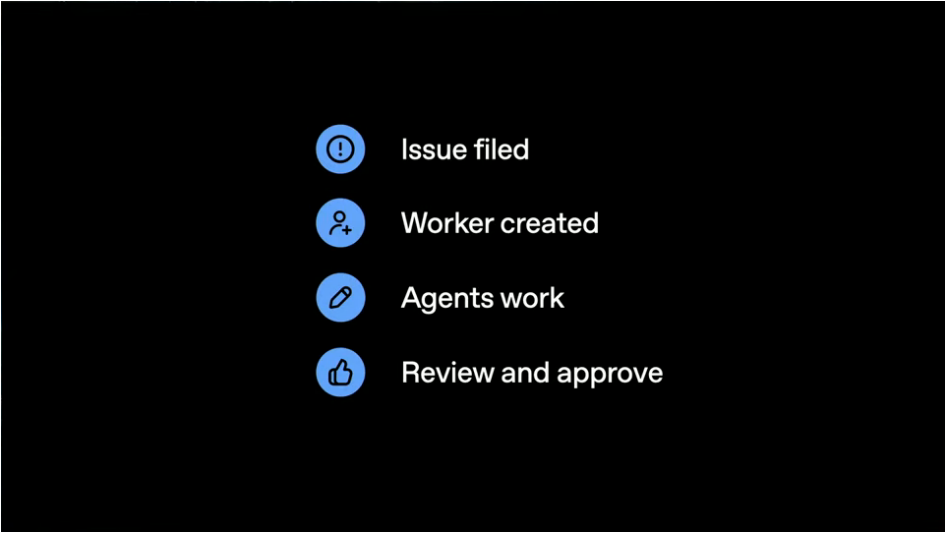


图 7: 画面用四个节点展示 issue 到审阅批准的流程：事件进入、工作智能体创建、智能体执行、人类审阅批准。⁷

原始幻灯片给出了四个外显步骤。为了让读者看清双循环边界，下面的重绘图把这四步展开成一个更完整的责任流：人先设定方向，管理器判断目标和上下文，工作智能体完成调查、实现和测试，管理器再把 PR、差异、构建或其他可审阅材料带回给人类审阅。检查通过后，循环继续进入下一轮。

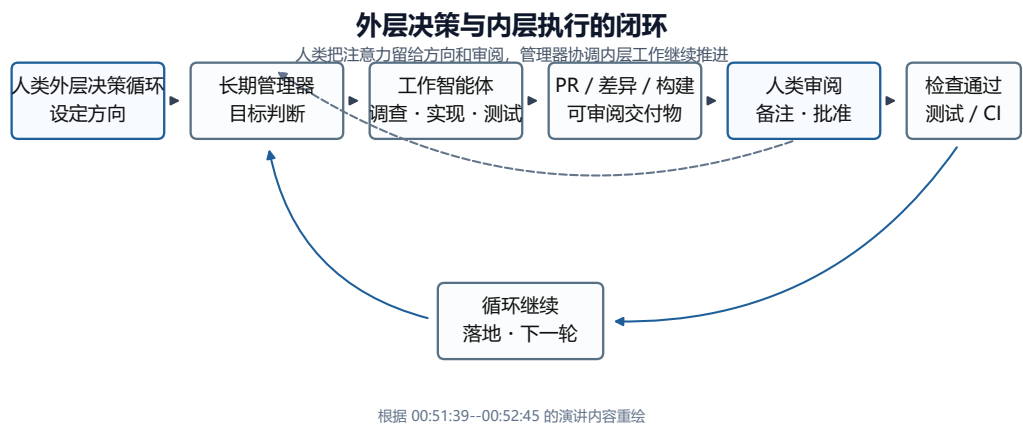


图 8: 根据 00:51:39–00:52:45 的演讲内容重绘的双循环总览图：内层执行由管理器和工作智能体推进，外层决策由人类设定方向、审阅证据并批准继续。

4.3 双循环边界如何保护注意力

内层执行循环包含调查、实现、测试和审查。这些步骤需要大量状态变化、日志、尝试和修正，适合由智能体与工作智能体完成。外层决策循环包含设定方向、判断是否符合项目目标、审阅差异、留下备注、批准或要求修改。这些步骤需要人类价值判断和项目判断，适合由人类完成。

这个分工像厨房里的出餐流程：后厨处理备料、烹饪、试味和装盘；主厨或店主检查是否符合

⁷ 视频画面时间区间：00:51:39–00:52:41

菜单定位、是否可以出餐、是否需要返工。顾客不需要看见每一次切菜，店主也不需要亲自执行每个动作；关键是出餐前的证据足够完整，责任边界足够清楚。

在 Peter 的语言里，智能体运行内层执行循环，人类设定方向并在外层循环做决定。这个边界让管理器成为注意力的过滤层：它不要求人类理解每一次临时选择，只要求人类在决策点看到原始问题、候选修改、证据材料和可验证结果。

4.4 本章小结

本节把“不要持续盯着智能体生成代码”转化为明确的工作流边界。内层执行循环由智能体完成，覆盖调查、实现、测试和审查；外层决策循环由人类完成，覆盖方向、审阅、备注、批准和异常处理。这个边界把注意力从中间消息转移到决策材料，使更好的循环能够在不消耗人类连续盯守的情况下继续推进。

5 长期运行的管理器：跨线程、宿主机/机器 (host/machine) 与工作入口

这一节的核心结论是：管理器一旦变成长期运行的协调智能体，它的边界就会自然越过单个应用窗口、单台电脑和单次会话。前几节已经把人的位置放到外层决策循环，把智能体放到内层执行循环；这里进一步说明执行场所和交互入口也需要被重新设计。长期管理器需要持续记住任务、在合适的计算环境中推进工作，并在需要人类判断时，从人实际所在的入口请求方向。

5.1 真实例证：Paul Solt 的定期唤醒 workflow

Peter Steinberger 先用 Paul Solt 的 Codex 工作流为例证。Paul Solt 固定了一个类似 *chief of staff* 的智能体：它每 10 分钟醒来一次，协调 GitHub 工作，并在需要额外方向时创建侧边栏里的线程。这里的线程指承载工作上下文、任务进展和可进入工作入口的工作线程。它让人不必一直守在终端前等待状态变化，只需要在工作需要额外 steering 时进入相应线程。

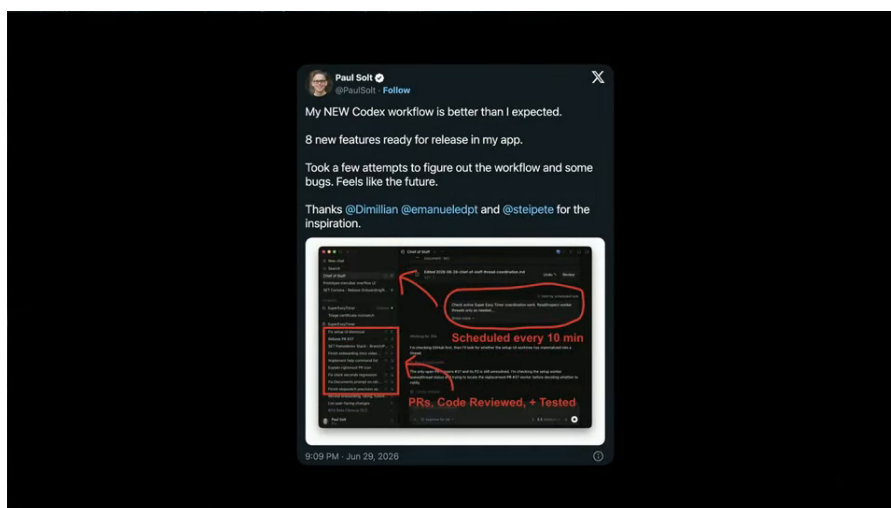


图 9: Paul Solt 的 Codex 工作流截图展示了“每 10 分钟调度一次”、PR、代码审查和测试状态。这张图把长期管理器的价值落到具体场景：它能周期性醒来、协调 GitHub 工作，并在需要人工方向时暴露可进入的线程。⁸

这个例子的关键点是定时器背后的工作承载方式：同一个管理器保留上下文，知道 GitHub 工作的状态，能在合适时机创建线程，并把人拉回到可决策的位置。触发器让工作继续前进，线程让人能够在需要时重新进入上下文，管理器负责把两者连接起来。

长期管理器的最低能力组合

长期管理器的最低能力组合是：保持持久上下文、按事件或时间被唤醒、创建可进入的线程、把工作结果带回给人审阅。缺少其中任何一项，系统都会退回到人手动轮询、手动路由或手动记忆的旧工作流。

5.2 线程与宿主机/机器解耦

接着，演讲把问题从“如何进入工作”推进到“工作在哪里执行”。当管理器长期存在，把它绑在某台 laptop 上就会显得狭窄。Codex 已经能在不同 hosts 之间移动工作；OpenClaw 也有 gateway and notes。它们都说明一个方向：线程可以承载上下文和任务入口，宿主机/机器（host/machine）则是任务实际运行或连接的计算环境，包括 cloud host、测试机器和本地机器。



图 10: 画面中的句子强调“线程不再绑定到机器”。这支撑本节的核心机制：线程负责承载工作，上下文和入口可以持续存在；宿主机/机器负责执行，能够按任务需要在 cloud 与 local machine 之间选择。⁹

把线程和宿主机/机器分开看，系统设计会更清晰。下表中的 Slack 工作入口指聊天协作工具中的人机交互入口，用来承接人的方向输入和管理器返回的判断材料：

维度	承载的问题	设计判断
线程	目标、上下文、进展、人工切入点	人从哪里恢复理解，何时需要 steering，哪些记录需要保留
宿主机/机器	代码运行、测试、构建、连接本地资源	哪些任务适合 cloud，哪些任务需要 local machine，哪些资源或权限限制执行位置
工作入口	sidebar、text、Slack 工作入口、其他协作界面	人当前在哪里，哪种提醒方式最少打断，哪种入口最适合做一次性审阅

⁸视频画面时间区间：00:52:45–00:53:08。

⁹视频画面时间区间：00:53:05–00:53:30。

这个拆分解释了 Steinberger 的实际诉求：他不想同步“自己在哪里工作”，希望智能体能够连接到任意机器，并知道哪些任务可放在 cloud 中完成，哪些任务需要本地机器。管理器的责任因此扩大为工作路由层：它既要理解任务，也要理解执行环境的差异。

5.3 跨入口交互：从应用会话到随处可 steer 的智能体

最后，演讲把管理器从应用窗口中释放出来。管理器不应困在某个 app 内部的 session 里；它应当是人可以发短信、从 Slack 工作入口 steer、或在任何地点接收反馈的智能体。Slack 工作入口在这里代表聊天协作界面中的人机交互入口：人可以在那里给方向，也可以在那里接收管理器带回的判断材料。



图 11: 画面用“One agent. Every surface.”概括跨入口形态：同一个智能体应能在多个交互表面上出现，管理器的连续性由持久上下文和线程维持，固定窗口只是一种入口。¹⁰

这里的“surface”可以理解为人类下达方向和接收结果的表面：sidebar 是开发工具里的表面，text 是移动通信表面，Slack 是团队协作表面。它们共享同一个管理器时，人的注意力负担会降低，因为人无需把不同入口里的状态手动拼回同一件工作。管理器替人维护连续性，人只在关键节点给方向、审阅结果或修正路线。

5.4 本章小结

长期运行的管理器把更好的循环推进到系统形态层面：

- 线程承载上下文、任务进展和人工切入点，避免把工作理解锁死在单次聊天或单个窗口里。
- 宿主机/机器承载实际执行环境，使同一项工作能根据权限、资源和本地依赖在 cloud 或 local machine 中运行。
- Slack 工作入口和 text 等表面让人从真实工作场景中 steer 管理器，减少在多个工具之间手动搬运状态的成本。
- 管理器的最终形态是一层持续存在的工作协调层：它把触发器、线程、执行环境和人类审阅连接成稳定闭环。

¹⁰ 视频画面时间区间：00:53:44–00:54:03。

6 总结与延伸

Steinberger 的收束讨论把整段演讲从工具演示提升为工程命题：这个问题还没有被解决。模型进步速度已经超过围绕模型的 harnesses 和 organizations，也就是超过了运行承载、权限边界、审阅流程、触发机制和组织协作方式的成熟速度。下一项工程问题已经转向承载系统设计，让模型、管理器、工作智能体和人类注意力配合成更好的循环（better loops）。

6.1 演讲的最终命题：设计承载系统

在最后半分钟里，speaker 提出两个连续判断。第一，当前系统还没有把长期管理器、跨机器执行、跨入口交互和人类审阅整合成最终形态。第二，设计这些承载系统就是下一项工程问题。这里的承载系统包括技术层和组织层：技术层要处理持久上下文、委派、触发器、宿主机/机器选择和审阅证据；组织层要决定谁设定目标、谁批准结果、异常如何升级、哪些自动化行为需要边界。

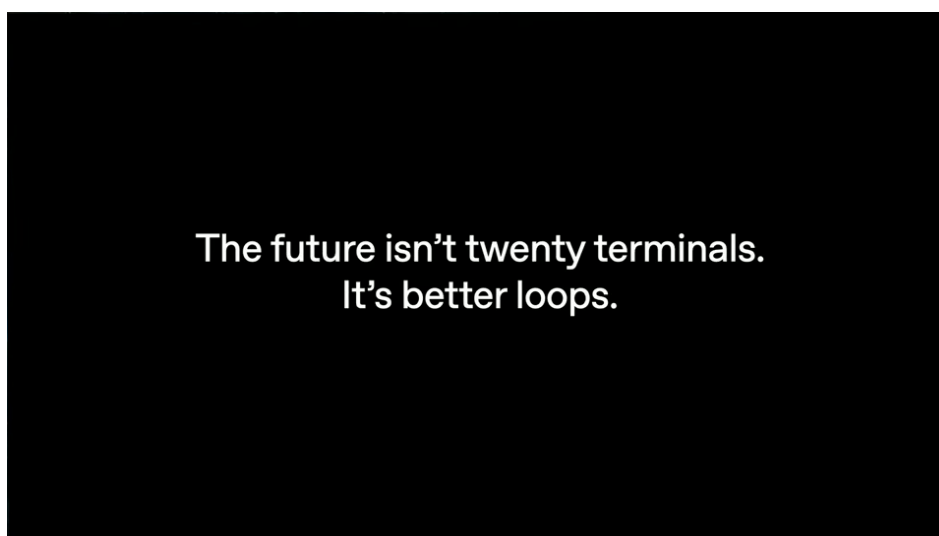


图 12: closing slide 将未来方向压缩为 better loops：重点从终端数量扩张转向循环质量、职责边界和注意力分配的工程设计。¹¹

这张收束图的重要性在于它把“二十个终端”的形象转化为反面参照：终端越多，人越像调度器、路由器和临时记忆；循环越好，人越能留在外层决策位置，把内层执行交给智能体系统。

6.2 PDF 的压缩结论：三项实践启发

第一，设计 loop。好的循环需要持久上下文保存任务记忆，需要委派把执行工作交给工作智能体，需要触发器在事件发生时唤醒同一个管理器。缺少闭环条件时，智能体仍然只能完成片段式任务。

第二，管理长期管理器。管理器在本主题中指长期运行的协调智能体，区别于人类组织职级。人类需要给它稳定目标、项目 notes、愿景边界、审阅标准和升级规则。管理器再把具体调查、实现、测试和审查分派给工作智能体，并在需要外层决策时返回 PR、diff、运行构建或其他可审阅证据。

第三，保护注意力。token 和 compute 能通过额度、硬件和并行执行缓解；注意力是人类认知资源，扩展方式更有限。更好的循环的价值在于减少人对中间消息、终端状态和机器位置的持续关注，把注意力保留给目标判断、风险审查和最终批准。

¹¹ 视频画面时间区间：00:54:18–00:54:30。

6.3 延伸：从工具清单转向 workflow 架构

把这段 keynote 用到真实团队时，最有价值的检查清单应围绕 workflow 架构展开，重点回答五个问题：

- 哪些上下文必须跨时间保留，哪些内容可以压缩成 notes 或审阅证据。
- 哪些任务适合管理器直接处理，哪些任务应该委派给工作智能体。
- 哪些事件应成为触发器，例如 issue 创建、测试失败、PR 更新或人工备注。
- 哪些执行必须在本地机器完成，哪些可以迁移到 cloud host 或测试机器。
- 人应该从哪些入口 steer 系统：开发工具侧边栏、text、Slack，还是其他团队协作表面。

因此，本主题的最终 takeaway 是：更好的循环是一种围绕人类注意力重新组织软件工程工作的方式。模型能力继续增长时，真正稀缺的会是能把模型能力接入可靠工作流的架构能力。下一阶段的软件工程问题，正是把长期管理器、工作智能体、跨机器执行和跨入口交互设计成可审阅、可恢复、可持续推进的闭环。

从本讲推导出的落地顺序

把这套思路应用到团队时，可以按四步建立最小闭环。第一，选择一个清晰事件作为触发器，例如 issue 创建、测试失败或 PR 更新。第二，指定长期管理器需要保留的项目 notes、目标边界和审阅标准。第三，规定工作智能体交回的证据包形式，例如 proposed diff、构建结果、测试日志或风险摘要。第四，把人工 steering 入口固定到团队真实使用的 surface 上，例如开发工具侧边栏、text 或 Slack 工作入口。这个顺序对应演讲中的责任分层：事件唤醒管理器，管理器委派工作智能体，工作智能体返回可审阅证据，人再用注意力完成外层决策。

应用边界

Steinberger 在结尾明确指出，模型前进速度已经超过 harnesses 和 organizations。团队落地时需要把这个判断翻译成边界条件：管理器可以推进执行循环，但权限、审阅责任和失败升级规则仍需由团队定义；跨机器执行可以提高灵活性，但本地依赖、凭据和资源隔离必须提前分层；跨入口 steering 可以减少切换成本，但每个入口都要回到同一条线程和同一份项目记忆。缺少这些边界时，更快的智能体只会放大协调问题。